

Projet de Partitionnement Robuste

Maël Verbois

2025-2026

1 Modélisation papier

1.1 Rappel des notations et du problème

Soit $G = (V, E)$ un graphe non orienté. Pour chaque arête $\{i, j\} \in E$, une distance l_{ij} est donnée. Pour chaque sommet $i \in V$, un poids w_i est associé. L'objectif est de partitionner les sommets V en au plus K sous-ensembles (parties) disjoints $V_1, \dots, V_K$ tels que le poids total de chaque partie ne dépasse pas B . On cherche à minimiser la somme des distances des arêtes dont les deux extrémités appartiennent à la même partie.

1.2 Modélisation par Programme Linéaire en Nombres Entiers (PLNE)

Nous introduisons les variables de décision suivantes :

- $x_{ik} \in \{0, 1\}$: vaut 1 si le sommet $i \in V$ est assigné à la partie $k \in \{1, \dots, K\}$, 0 sinon.
- $y_{ij} \in \{0, 1\}$: vaut 1 si l'arête $\{i, j\} \in E$ a ses deux extrémités dans la même partie, 0 sinon.

Le problème statique peut être modélisé par le programme linéaire suivant :

$$\text{Minimiser} \quad \sum_{\{i,j\} \in E} l_{ij} y_{ij} \quad (1)$$

$$\text{Sous contraintes} \quad \sum_{k=1}^K x_{ik} = 1, \quad \forall i \in V \quad (2)$$

$$\sum_{i \in V} w_i x_{ik} \leq B, \quad \forall k \in \{1, \dots, K\} \quad (3)$$

$$y_{ij} \geq x_{ik} + x_{jk} - 1, \quad \forall \{i, j\} \in E, \forall k \in \{1, \dots, K\} \quad (4)$$

$$x_{ik} \in \{0, 1\}, \quad y_{ij} \in \{0, 1\} \quad \forall i \in V, \forall \{i, j\} \in E, \forall k \quad (5)$$

Explication des contraintes

1. La contrainte (2) assure que chaque sommet est affecté à exactement une partie.
2. La contrainte (3) assure que la somme des poids des sommets d'une partie k ne dépasse pas la capacité B .
3. La contrainte (4) lie les variables de sommets et d'arêtes. Si les sommets i et j sont tous deux dans la partie k , alors $x_{ik} = 1$ et $x_{jk} = 1$. La contrainte devient $y_{ij} \geq 1 + 1 - 1 = 1$, forçant y_{ij} à 1. Puisque l'objectif est de minimiser une somme à coefficients positifs ($l_{ij} \geq 0$), y_{ij} prendra la valeur 0 si i et j ne sont jamais dans la même partie.

Preuve de la validité de la modélisation

Il existe une bijection entre les solutions optimales du problème de partitionnement et celles de ce PLNE.

- **Sens direct** : Soit une partition admissible optimale. On fixe $x_{ik} = 1$ si i est dans la partie k 0 sinon. Les contraintes (2) et (3) sont satisfaites par définition. Pour chaque arête $\{i, j\}$, si i et j sont dans la même partie, on fixe $y_{ij} = 1$, sinon $y_{ij} = 0$ cela garantit la contrainte (4) . La valeur de l'objectif correspond exactement à la somme des distances intra-classes d'après la remarque 3 et elle est bien minimum par optimalité de la partition initiale (les solutions admissibles sont des partitions valides cf réciproque).
- **Sens réciproque** : Soit (x, y) une solution optimale du PLNE. Les x_{ik} définissent une partition valide (chaque sommet est assigné, capacité respectée). La contrainte (4) implique que si $x_{ik} = x_{jk} = 1$, alors $y_{ij} \geq 1$. Comme on minimise $\sum l_{ij}y_{ij}$ avec $l_{ij} \geq 0$, on fixera $y_{ij} = 0$ si la contrainte ne le force pas à 1. Ainsi, $y_{ij} = 1$ si et seulement si i et j sont dans la même partie. L'objectif représente bien le coût de la partition et est donc optimale par optimalité de la solution optimale et le fait que les partitions valides sont des solutions valides d'après le sens direct.

Preuve de la compacité

Une modélisation est dite compacte si le nombre de variables et de contraintes est polynomial en la taille de l'instance.

- **Variables** : Il y a $|V| \times K$ variables x_{ik} et $|E|$ variables y_{ij} .
- **Contraintes** : Il y a $|V|$ contraintes d'affectation, K contraintes de capacité, et $|E| \times K$ contraintes de lien.

Comme le nombre de contraintes dépend de K le problème n'est à priori pas compact. Cependant on peut s'intéresser uniquement aux partitions avec des parties non vides et dans ce cas, nous avons $K \leq |V|$ (dans le pire des cas chaque sommet correspond à une partie). Le nombre total de variables est en $O(|V|^2 + |E|)$ et le nombre de contraintes est en $O(|E||V|)$. Ces quantités étant polynomiales par rapport à la taille du graphe, la modélisation est compacte.

1.3 Modélisation du problème robuste

Définition des ensembles d'incertitude

D'après le sujet, les distances incertaines appartiennent à l'ensemble $\mathcal{U}^1$ défini par les variables de déviation δ_{ij}^1 :

$$\mathcal{U}^1 = \left\{ (l_{ij}^1 = l_{ij} + \delta_{ij}^1(\hat{l}_i + \hat{l}_j))_{ij \in E} \mid \sum_{\{i,j\} \in E} \delta_{ij}^1 \leq L, \quad 0 \leq \delta_{ij}^1 \leq 3 \quad \forall \{i, j\} \in E \right\}$$

Les poids incertains appartiennent à l'ensemble $\mathcal{U}^2$ défini par les variables de déviation δ_i^2 :

$$\mathcal{U}^2 = \left\{ (w_i^2 = w_i(1 + \delta_i^2))_{i \in V} \mid \sum_{i \in V} \delta_i^2 \leq W, \quad 0 \leq \delta_i^2 \leq W_D \quad \forall i \in V \right\}$$

Programme Robuste

Le problème robuste consiste à trouver une solution (x, y) réalisable pour *toutes* les réalisations possibles des poids dans $\mathcal{U}^2$, et qui minimise le coût dans le *pire* scénario de distances dans $\mathcal{U}^1$. On se permettra l'abus de notation qui confond les variable robustes avec leur paramétrisation dans les ensembles $\mathcal{U}^1$ et $\mathcal{U}^2$.

La formulation est la suivante :

$$\min_{x,y} \max_{\delta^1 \in \mathcal{U}^1} \sum_{\{i,j\} \in E} \left(l_{ij} + \delta_{ij}^1 (\hat{l}_i + \hat{l}_j) \right) y_{ij} \quad (6)$$

$$\text{Sous contraintes} \quad \sum_{k=1}^K x_{ik} = 1, \quad \forall i \in V \quad (7)$$

$$\sum_{i \in V} w_i (1 + \delta_i^2) x_{ik} \leq B, \quad \forall k \in \{1, \dots, K\}, \forall \delta^2 \in \mathcal{U}^2 \quad (8)$$

$$y_{ij} \geq x_{ik} + x_{jk} - 1, \quad \forall \{i, j\} \in E, \forall k \quad (9)$$

$$x_{ik} \in \{0, 1\}, \quad y_{ij} \in \{0, 1\} \quad (10)$$

1.4 Résolution par plans coupants et LazyCallback

3.a) Reformulation avec variable épigraphique

Nous introduisons une variable réelle continue η représentant la valeur de l'objectif dans le pire cas.

$$\text{Minimiser} \quad \eta \quad (11)$$

$$\text{Sous contraintes} \quad \eta \geq \sum_{\{i,j\} \in E} \left(l_{ij} + \delta_{ij}^1 (\hat{l}_i + \hat{l}_j) \right) y_{ij}, \quad \forall \delta^1 \in \mathcal{U}^1 \quad (12)$$

$$\sum_{i \in V} w_i (1 + \delta_i^2) x_{ik} \leq B, \quad \forall k \in \{1, \dots, K\}, \forall \delta^2 \in \mathcal{U}^2 \quad (13)$$

$$\sum_{k=1}^K x_{ik} = 1, \quad \forall i \in V \quad (14)$$

$$y_{ij} \geq x_{ik} + x_{jk} - 1, \quad \forall \{i, j\} \in E, \forall k \quad (15)$$

$$x_{ik} \in \{0, 1\}, \quad y_{ij} \in \{0, 1\}, \quad \eta \in \mathbb{R} \quad (16)$$

Donc, le problème maître restreint des plans coupants correspond exactement aux problèmes précédents où l'on remplace $\mathcal{U}^1, \mathcal{U}^2$ par deux sous ensembles finis $\mathcal{U}^{1*}, \mathcal{U}^{2*}$.

3.b) Ensembles initiaux $\mathcal{U}^{1*}$ et $\mathcal{U}^{2*}$

Pour initialiser l'algorithme de plans coupants, nous choisissons les scénarios du problème statique qui correspond au scénario le plus optimiste.

- $\mathcal{U}^{1*} = \{1\}$, c'est-à-dire le scénario où $\delta_{ij}^1 = 0$ pour tout $\{i, j\} \in E$.
- $\mathcal{U}^{2*} = \{\mathbf{w}\}$, c'est-à-dire le scénario où $\delta_i^2 = 0$ pour tout $i \in V$.

Ainsi, la première itération du problème maître résout exactement le problème statique défini à la section 2.1.

3.c) Sous-problèmes de séparation

Soit $(\hat{x}, \hat{y}, \hat{\eta})$ une solution optimale courante du problème maître restreint.

Sous-problème 1 : Nous cherchons à maximiser le coût total des arêtes sélectionnées en jouant sur les variables d'incertitude δ^1 . Comme la partie $\sum l_{ij} \hat{y}_{ij}$ est constante, cela revient à maximiser le terme de déviation :

$$\begin{aligned} SP1(\hat{y}) = \max_{\delta^1} \quad & \sum_{\{i,j\} \in E} \left(\hat{y}_{ij} (\hat{l}_i + \hat{l}_j) \right) \delta_{ij}^1 \\ \text{s.c.} \quad & \sum_{\{i,j\} \in E} \delta_{ij}^1 \leq L \\ & 0 \leq \delta_{ij}^1 \leq 3, \quad \forall \{i, j\} \in E \end{aligned}$$

Ce problème est un problème de sac à dos continu. Il peut se résoudre très efficacement (algorithme glouton) en triant les coefficients $\hat{y}_{ij}(\hat{l}_i + \hat{l}_j)$ par ordre décroissant et en saturant les δ_{ij}^1 en suivant cette ordre.

Sous-problème 2 : Ce problème se décompose par partie. Pour chaque partie $k \in \{1, \dots, K\}$, nous cherchons le poids total maximal en jouant sur δ^2 . Pour une partie k fixée :

$$\begin{aligned} SP2_k(\hat{x}) = \max_{\delta^2} \quad & \sum_{i \in V} (w_i \hat{x}_{ik}) \delta_i^2 \\ \text{s.c.} \quad & \sum_{i \in V} \delta_i^2 \leq W \\ & 0 \leq \delta_i^2 \leq W_D, \quad \forall i \in V \end{aligned}$$

Remarque : Le terme constant $\sum w_i \hat{x}_{ik}$ n'intervient pas dans l'optimisation mais sera ajouté au résultat pour vérifier la contrainte B . De même que pour SP1, c'est un sac à dos continu résoluble par tri des coefficients $w_i \hat{x}_{ik}$.

3.d) Conditions d'optimalité

Une solution $(\hat{x}, \hat{y}, \hat{\eta})$ du problème maître est optimale pour le problème robuste complet si elle satisfait les conditions suivantes :

1. **Validité de l'objectif :** La valeur $\hat{\eta}$ (estimation du coût par le maître) est supérieure ou égale au coût réel dans le pire cas identifié par le sous-problème 1.

$$\hat{\eta} \geq \sum_{\{i,j\} \in E} l_{ij} \hat{y}_{ij} + SP1(\hat{y})$$

2. **Admissibilité robuste :** Pour chaque partie k , le poids total dans le pire cas identifié par le sous-problème 2 respecte la borne B .

$$\sum_{i \in V} w_i \hat{x}_{ik} + SP2_k(\hat{x}) \leq B, \quad \forall k \in \{1, \dots, K\}$$

Si ces conditions sont remplies, alors $(\hat{x}, \hat{y})$ est une solution réalisable robuste et sa valeur objective est minimale car la valeur du problème maître est un minorant de la valeur du problème robuste qui est aussi une solution admissible.

3.e) Expression des coupes

Si les conditions ci-dessus ne sont pas remplies, nous générons des coupes à ajouter au problème maître. Soient δ^{1*} la solution optimale de SP1 et $\delta^{2*,k}$ la solution optimale de SP2 pour la partie k .

- **Coupe d'optimalité :** Si $\hat{\eta} < \sum l_{ij} \hat{y}_{ij} + SP1(\hat{y})$, nous ajoutons la coupe suivante au problème maître (ajout du scénario δ^{1*} à $\mathcal{U}^{1*}$) :

$$\eta \geq \sum_{\{i,j\} \in E} \left(l_{ij} + \delta_{ij}^{1*} (\hat{l}_i + \hat{l}_j) \right) y_{ij}$$

- **Coupe de faisabilité :** Si pour une partie k , $\sum w_i \hat{x}_{ik} + SP2_k(\hat{x}) > B$, nous ajoutons la coupe suivante au problème maître (ajout du scénario $\delta^{2*,k}$ à $\mathcal{U}^{2*}$) :

$$\sum_{i \in V} w_i (1 + \delta_i^{2*,k}) x_{ik} \leq B$$

1.5 Résolution par dualisation

4.a) Reformulation de l'objectif

L'objectif du problème robuste est :

$$\min_{x,y} \max_{\delta^1 \in \mathcal{U}^1} \sum_{\{i,j\} \in E} (l_{ij} + \delta_{ij}^1(\hat{l}_i + \hat{l}_j))y_{ij}$$

On la réécrit en deux parties dont une dépend de δ^1 :

$$\min_{x,y} \left(\sum_{\{i,j\} \in E} l_{ij}y_{ij} + \max_{\delta^1 \in \mathcal{U}^1} \sum_{\{i,j\} \in E} \delta_{ij}^1(\hat{l}_i + \hat{l}_j)y_{ij} \right)$$

4.b) Problème interne lié à δ_{ij}^1

Pour des variables y_{ij} fixées, le terme de maximisation correspond au PL suivant (P1) :

$$\begin{aligned} (P1) \quad & \max_{\delta^1} \sum_{\{i,j\} \in E} ((\hat{l}_i + \hat{l}_j)y_{ij}) \delta_{ij}^1 \\ \text{s.c.} \quad & \sum_{\{i,j\} \in E} \delta_{ij}^1 \leq L \\ & \delta_{ij}^1 \leq 3, \quad \forall \{i,j\} \in E \\ & \delta_{ij}^1 \geq 0, \quad \forall \{i,j\} \in E \end{aligned}$$

4.c) Dualisation de ce problème

Associons les variables duales suivantes aux contraintes de (P1) :

- $\sigma^1 \geq 0$ associée à la contrainte de budget global $\sum \delta_{ij}^1 \leq L$.
- $\mu_{ij}^1 \geq 0$ associée à chaque contrainte de borne supérieure $\delta_{ij}^1 \leq 3$.

Le problème dual (D1) s'écrit alors :

$$\begin{aligned} (D1) \quad & \min_{\sigma^1, \mu^1} L\sigma^1 + \sum_{\{i,j\} \in E} 3\mu_{ij}^1 \\ \text{s.c.} \quad & \sigma^1 + \mu_{ij}^1 \geq (\hat{l}_i + \hat{l}_j)y_{ij}, \quad \forall \{i,j\} \in E \\ & \sigma^1 \geq 0, \quad \mu_{ij}^1 \geq 0 \end{aligned}$$

D'après le théorème de la dualité forte, la valeur optimale de (P1) est égale à celle de (D1). Nous pourrions donc remplacer le "max" dans la fonction objectif globale par ce problème de minimisation.

4.d) Reformulation des contraintes robustes

La contrainte de capacité pour la partie k doit être satisfaite pour tout scénario de $\mathcal{U}^2$. Cela équivaut à exiger que le poids maximal de la partie k ne dépasse pas B :

$$\max_{\delta^2 \in \mathcal{U}^2} \left(\sum_{i \in V} w_i(1 + \delta_i^2)x_{ik} \right) \leq B$$

En isolant le terme incertain, on obtient :

$$\sum_{i \in V} w_i x_{ik} + \max_{\delta^2 \in \mathcal{U}^2} \sum_{i \in V} (w_i x_{ik}) \delta_i^2 \leq B$$

4.e) Problème interne lié aux variables δ_i^2

Pour une partie k et un vecteur x fixés, le terme de maximisation correspond au problème linéaire $(P2_k)$,

$$\begin{aligned}
 (P2_k) \quad & \max_{\delta^2} \quad \sum_{i \in V} (w_i x_{ik}) \delta_i^2 \\
 \text{s.c.} \quad & \sum_{i \in V} \delta_i^2 \leq W & (\text{variable duale } \sigma_k^2) \\
 & \delta_i^2 \leq W_i, \quad \forall i \in V & (\text{variable duale } \lambda_{ik}^2) \\
 & \delta_i^2 \geq 0, \quad \forall i \in V
 \end{aligned}$$

4.f) Dualisation de ces problèmes

Le problème dual $(D2_k)$ s'écrit :

$$\begin{aligned}
 (D2_k) \quad & \min_{\sigma_k^2, \lambda_{ik}^2} \quad W \sigma_k^2 + \sum_{i \in V} W_i \lambda_{ik}^2 \\
 \text{s.c.} \quad & \sigma_k^2 + \lambda_{ik}^2 \geq w_i x_{ik}, \quad \forall i \in V \\
 & \sigma_k^2 \geq 0, \quad \lambda_{ik}^2 \geq 0
 \end{aligned}$$

Par le théorème de la dualité forte en PL on peut remplacer le PL par sa dualisation dans le modèle robuste.

4.g) Programme Linéaire en Nombres Entiers Final

En remplaçant les maximisations par les minimisations de leurs problèmes duaux respectifs (théorème de dualité forte de la PL), le problème robuste se réécrit ainsi :

Reformulation : L'objectif devient :

$$\min_{x, y} \left(\sum_{\{i, j\} \in E} l_{ij} y_{ij} + \min_{\sigma^1, \mu^1} \left(L \sigma^1 + \sum_{\{i, j\} \in E} 3 \mu_{ij}^1 \right) \right)$$

Les contraintes de capacité robustes deviennent :

$$\sum_{i \in V} w_i x_{ik} + \min_{\sigma_k^2, \lambda_{ik}^2} \left(W \sigma_k^2 + \sum_{i \in V} W_i \lambda_{ik}^2 \right) \leq B, \quad \forall k \in \{1, \dots, K\}$$

En regroupant les minimums dans l'objectif et en remplaçant le minimum dans la contrainte robuste par l'existence d'une solution réalisable (ce qui est équivalent). On obtient le programme suivant :

Programme Linéaire Mixte compact :

$$\text{Min} \quad \sum_{\{i, j\} \in E} l_{ij} y_{ij} + L \sigma^1 + \sum_{\{i, j\} \in E} 3 \mu_{ij}^1 \quad (17)$$

$$\text{S.c.} \quad \sum_{k=1}^K x_{ik} = 1, \quad \forall i \in V \quad (18)$$

$$y_{ij} \geq x_{ik} + x_{jk} - 1, \quad \forall \{i, j\} \in E, \forall k \quad (19)$$

$$\sigma^1 + \mu_{ij}^1 \geq (\hat{l}_i + \hat{l}_j) y_{ij}, \quad \forall \{i, j\} \in E \quad (20)$$

$$\sum_{i \in V} w_i x_{ik} + W \sigma_k^2 + \sum_{i \in V} W_i \lambda_{ik}^2 \leq B, \quad \forall k \quad (21)$$

$$\sigma_k^2 + \lambda_{ik}^2 \geq w_i x_{ik}, \quad \forall i \in V, \forall k \quad (22)$$

$$x_{ik}, y_{ij} \in \{0, 1\} \quad (23)$$

$$\sigma^1, \mu_{ij}^1, \sigma_k^2, \lambda_{ik}^2 \geq 0 \quad (24)$$

En notant $N = |V|$ (sommets) et $M = |E|$ (arêtes), le modèle comporte exactement :

- **Variables** : $2NK + 2M + K + 1$.
- **Contraintes** : $NK + MK + N + M + K$.

Comme $K \leq N$, la taille du problème reste polynomiale.

2 Résolution Numérique et Analyse

Cette section présente l'architecture logicielle développée, les stratégies de bornes inférieures pour évaluer la qualité des solutions, ainsi qu'une comparaison des méthodes de résolution exactes et heuristiques. On trouvera en annexe les résultats pour toutes les instances ainsi qu'une visualisation des solutions statiques et robustes trouvées. Les stratégies sont les suivantes :

- Méthode de résolution par plans coupants
- Méthode par Branch and Cut et LazyCallback
- Méthode par Dualisation des sous problèmes robustes
- Méthode heuristique par Génération de Colonnes
- Méthode heuristique Relaxed-Repair et comparaison avec bornes inférieures combinatoires

2.1 Architecture Logicielle et Protocoles

L'ensemble des algorithmes a été implémenté en langage **Julia**, utilisant le package **JuMP** pour la modélisation mathématique et le solveur **Gurobi** pour la résolution des PLNE. Toutes les solutions ont été testées avec un temps limite de 300s.

Une architecture modulaire centrée sur un module principal nommé **Solver** a été choisi. Ce choix de conception permet de découpler la définition des données du problème des stratégies de résolution.

- **Structure de Données (ProblemData)** : Encapsule de manière immuable les paramètres du graphe $G = (V, E)$, les matrices de distances nominales, les poids des sommets ainsi que les paramètres d'incertitude.
- **Interface** : Une interface commune permet d'appeler indifféremment les méthodes de plans coupants, la dualisation ou la génération de colonnes. Cela garantit que toutes les méthodes bénéficient des mêmes routines de chargement d'instance et de vérification de solution.
- **Gestion des Résultats** : Chaque algorithme renvoie une structure standardisée **ResultatAlgorithme** contenant le statut final, la meilleure solution entière trouvée (UB), la meilleure borne inférieure prouvée (LB), le gap relatif et le temps de calcul.

On trouvera dans le repo github un tutoriel indiquant la marche à suivre pour faire fonctionner le code.

2.2 Analyse du Problème Statique

Avant d'aborder la robustesse, l'analyse du problème statique révèle des difficultés intrinsèques liées à la modélisation choisie. Si les petites instances du PLNE ($|V| < 20$) sont résolues instantanément par Gurobi, les instances de taille moyenne ($|V| \geq 30$) présentent une convergence lente voir inexistante. En cause, l'incapacité du solveur à trouver une borne inférieure de qualité permettant de fermer l'instance.

Deux facteurs principaux expliquent ce phénomène :

1. **Symétries** : Toute permutation des indices des sous-ensembles $V_1, \dots, V_K$ représente la même solution physique. Bien que des contraintes de rupture de symétrie (ex: classer les partitions par taille ou par index du plus petit élément) aient été testées, elles alourdissent le modèle sans arriver à faire mieux que les méthodes natives du solveur.

2. **Faiblesse de la Relaxation Continue :** La relaxation linéaire du modèle compact est faible. Il est "peu coûteux" pour le solveur de satisfaire les contraintes d'affectation $\sum_k x_{ik} = 1$ en fixant $x_{ik} = 1/K$ pour tout k , tout en maintenant les variables de lien y_{ij} à 0, minimisant ainsi artificiellement l'objectif. Il a été tenté d'ajouter des inégalités valides (inégalités triangulaires) de la forme :

$$y_{ij} + y_{jk} - y_{ik} \leq 1 \quad \forall i, j, k \in V$$

Ces contraintes, qui imposent la transitivité de la relation "être dans la même partie" (si i et j sont liés et que j et k sont liés, alors i et k doivent l'être), renforcent l'enveloppe convexe. Cependant, leur nombre en $O(|V|^3)$ alourdit considérablement le modèle sans parvenir à compenser efficacement la tendance de la relaxation à fractionner les variables d'affectation.

On s'attend donc à deux phénomènes pour toutes les méthodes:

- Un allongement significatif du temps de calcul lorsque K augmente lié à l'augmentation des symétries et une augmentation de la combinatoire du problème (plus de partitionnement possible).
- Une relaxation de moins bonne qualité et donc des gaps plus mauvais pour les instances avec des valeurs de K plus grande.

Quelques résultats

Table 1: Résultats Statiques (Sélection)

Instance	Objectif	Gap (%)	Temps (s)
22_ulysses_3.tsp	284.21	0	0.42
22_ulysses_6.tsp	82.63	0	8.82
22_ulysses_9.tsp	26.22	0	2.27
52_berlin_3.tsp	159456.39	51	300.08
52_berlin_6.tsp	51862.15	87	300.02
52_berlin_9.tsp	25810.92	93	300.03
318_lin_3.tsp	18671783.98	50	300.55
318_lin_6.tsp	5833529.63	100	300.57
318_lin_9.tsp	2929125.77	100	300.28

2.3 Bornes Inférieures

Face à la faiblesse de la borne inférieure fournie par le MIP, on décide dans un premier temps d'obtenir des bornes inférieures valides par des méthodes spécifiques pour certifier la qualité des solutions futures. On peut commencer par remarquer que comme le scénario statique appartient à l'ensemble des scénarios du robuste, toute borne inférieure pour le statique est une borne inférieure pour le robuste (relaxation des contraintes de poids robustes et définition du max sur les scénario de coût).

2.3.1 Borne Combinatoire par Cardinalité

Cette méthode repose sur une propriété structurelle simple : pour partitionner un ensemble de sommets en K sous-ensembles, le nombre total d'arêtes internes est minimisé lorsque la répartition des effectifs est la plus équilibrée possible.

Preuve par argument d'échange. Considérons une partition contenant deux sous-ensembles de tailles respectives n_a et n_b . Le nombre d'arêtes internes à ces deux groupes est la somme des paires distinctes possibles dans chacun d'eux. Supposons qu'il existe un déséquilibre significatif entre ces deux groupes, tel que :

$$n_a \geq n_b + 2$$

Si l'on transfère un sommet du groupe le plus grand (taille n_a) vers le groupe le plus petit (taille n_b), la variation du nombre total d'arêtes internes Δ correspond à la différence entre les liens créés dans le nouveau groupe et les liens supprimés dans l'ancien :

$$\Delta = n_b - (n_a - 1)$$

Puisque par hypothèse $n_a > n_b + 1$, cette variation Δ est strictement négative. Par conséquent, tant que la différence de taille entre deux parties est supérieure ou égale à 2, il est toujours possible de réduire le nombre total d'arêtes par un simple transfert. L'optimum est donc atteint uniquement lorsque les tailles des cliques sont quasi-identiques (c'est-à-dire égales à $\lfloor n/K \rfloor$ ou $\lceil n/K \rceil$).

Calcul de la borne. Une fois ce nombre minimal théorique d'arêtes déterminé, noté M_{min} , nous pouvons ignorer la structure du graphe et nous concentrer uniquement sur les coûts. La borne inférieure valide LB_{card} est obtenue en sommant simplement les poids des M_{min} arêtes les plus légères du graphe G .

2.3.2 Relaxation par partition en pseudo-clique

La borne par cardinalité présentée précédemment ne respecte pas la structure locale des cliques : elle sélectionne les arêtes les moins chères sans vérifier si elles sont cohérentes pour former une partition et des cliques. Pour obtenir une borne plus forte, on exploite une propriété locale d'une clique : dans une clique de taille t , chaque sommet possède nécessairement un degré égal à $t - 1$.

Cas Statique

Nous définissons une *pseudo-clique* de taille t comme un ensemble de t sommets auxquels on impose un degré égal à $t - 1$, sans exiger la connectivité complète (les arêtes peuvent relier ces sommets à n'importe quels autres du graphe). Dans le cas statique, le problème relaxé revient à choisir la taille t de la pseudo-clique à laquelle chaque sommet appartient afin de minimiser le coût total des arêtes, sous la contrainte que les degrés soient cohérents. On peut l'écrire sous la forme d'un PLNE.

Soit $z_{it} \in \{0, 1\}$ valant 1 si le sommet i est assigné à une pseudo-clique de taille t , $x_{ij} \in \{0, 1\}$ valant 1 si l'arête ij est sélectionnée ou non, Le modèle s'écrit :

$$\begin{aligned} \min \quad & \sum_{\{i,j\} \in E} l_{ij} x_{ij} \\ \text{s.c.} \quad & \sum_{j \neq i} x_{ij} = \sum_{t=1}^n (t-1) z_{it} \quad \forall i \in V \\ & \sum_{t=1}^n z_{it} = 1 \quad \forall i \in V \end{aligned}$$

Cette relaxation est bien une borne inférieure car toute partition en vraies cliques respecte ces contraintes de degrés. Un avantage majeur de cette relaxation est que l'on peut ajouter les contraintes spécifiques de notre problème sur le budget des partitions et leur nombre en adaptant le modèle, tout en évitant les contraintes quadratiques liant les arêtes et les sommets. En introduisant une variable entière N_t représentant le nombre de pseudo-cliques de taille t utilisées, nous pouvons lier l'affectation des sommets à la capacité B et fixer le nombre de partition à K :

Le Programme Linéaire en Nombres Entiers complet s'écrit :

$$\begin{aligned} \min \quad & \sum_{\{i,j\} \in E} l_{ij} x_{ij} \\ \text{s.c.} \quad & \sum_{j \neq i} x_{ij} = \sum_{t=1}^n (t-1) z_{it} & \forall i \in V \quad (\text{Contrainte de Degrés}) \\ & \sum_{t=1}^n z_{it} = 1 & \forall i \in V \quad (\text{Affectation Unique}) \\ & \sum_{i \in V} z_{it} = t \cdot N_t & \forall t \in \{1, \dots, n\} \quad (\text{Cohérence Taille}) \\ & \sum_{t=1}^n N_t = K & (\text{Nombre de Partitions}) \\ & \sum_{i \in V} w_i z_{it} \leq B \cdot N_t & \forall t \in \{1, \dots, n\} \quad (\text{Capacité Relaxée}) \\ & x_{ij}, z_{it} \in \{0, 1\}, N_t \in \mathbb{N} \end{aligned}$$

Extension au Cas Robuste.

Pour le problème robuste, nous enrichissons ce modèle en intégrant les ensembles d'incertitude U^1 (distances) et U^2 (poids). On remarque que dans notre modélisation, il est impossible de distinguer les pseudo-cliques de même taille les uns des autres. On ne peut donc pas calculer aisément le coût robuste d'une pseudo-clique individuelle. Néanmoins, on peut calculer le coût robuste de l'ensemble des pseudo-cliques de taille t pour un t fixé en choisissant le pire scénario pour l'ensemble de ces pseudo-cliques (et non le pire par pseudo-clique).

C'est une relaxation du problème robuste car nous remplaçons la contrainte locale "chaque cluster robuste k doit respecter la capacité B " par une contrainte agrégée "la somme des poids des N_t clusters robuste de taille t ne doit pas dépasser la capacité totale $N_t \times B$ ". Si une solution respecte les contraintes robustes individuelles, elle respecte la contrainte globale, c'est une relaxation.

Nous introduisons les variables duales pour gérer l'incertitude : σ^1, μ_{ij}^1 pour les distances (budget global L) et S_t^2, Λ_{it}^2 pour les poids (budget local W agrégé par taille).

Le Programme Linéaire Mixte complet s'écrit :

$$\begin{aligned}
\min \quad & \sum_{\{i,j\} \in E} l_{ij} x_{ij} + L\sigma^1 + \sum_{\{i,j\} \in E} 3\mu_{ij}^1 \\
\text{s.c.} \quad & \sum_{j \neq i} x_{ij} = \sum_{t=1}^n (t-1) z_{it} & \forall i \in V \quad (\text{Degrés}) \\
& \sum_{t=1}^n z_{it} = 1 & \forall i \in V \quad (\text{Affectation}) \\
& \sum_{i \in V} z_{it} = t \cdot N_t & \forall t \in \{1, \dots, n\} \quad (\text{Cohérence}) \\
& \sum_{t=1}^n N_t = K & (\text{Total Partitions}) \\
& \sigma^1 + \mu_{ij}^1 \geq (\hat{l}_i + \hat{l}_j) x_{ij} & \forall \{i, j\} \in E \quad (\text{Dualité Distances}) \\
& \sum_{i \in V} w_i z_{it} + W S_t^2 + \sum_{i \in V} W_i \Lambda_{it}^2 \leq B \cdot N_t & \forall t \in \{1, \dots, n\} \quad (\text{Capacité Robuste Agrégée}) \\
& S_t^2 + \Lambda_{it}^2 \geq w_i z_{it} & \forall i \in V, \forall t \quad (\text{Dualité Poids}) \\
& x_{ij}, z_{it} \in \{0, 1\}, N_t \in \mathbb{N} \\
& \sigma^1, \mu^1, S^2, \Lambda^2 \geq 0
\end{aligned}$$

Ce modèle capture simultanément la structure nécessaire (degrés) et le coût de la robustesse, fournissant une meilleure borne LB_{deg} que précédemment.

Quelques Résultats

Table 2: Comparaison des Bornes Inférieures (Sélection)

Instance	Combinatoire		Pseudo-Clique Statique		Pseudo-Clique Robuste	
	LB	T(s)	LB	T(s)	LB	T(s)
22_ulysses_3.tsp	179.46	0.00	260.31	0.11	281.15	0.74
22_ulysses_6.tsp	44.83	0.00	70.07	0.06	78.50	0.15
22_ulysses_9.tsp	16.80	0.00	25.64	0.04	33.95	0.12
52_berlin_3.tsp	93805.24	0.00	127236.58	0.51	141106.96	24.68
52_berlin_6.tsp	26663.76	0.00	39138.66	0.25	50460.28	30.78
52_berlin_9.tsp	12488.95	0.00	19720.90	0.29	30555.75	198.31
318_lin_3.tsp	14454856.36	0.00	15183928.61	302.28	15414696.74	305.47
318_lin_6.tsp	4666367.94	0.00	4887027.74	161.73	5103538.44	304.42
318_lin_9.tsp	2390541.01	0.00	2510929.77	59.21	2709730.92	304.34

On utilisera ensuite ces bornes inférieures pour juger de la qualité des solutions et/ou des bornes inférieures trouvées par les différentes méthodes.

2.4 Comparaison des Méthodes Robustes Compactes

On compare les trois approches exactes demandées pour résoudre le problème robuste.

2.4.1 Plans Coupants Naïfs

Cette méthode résout itérativement le problème maître statique. À chaque optimum entier trouvé, on vérifie si les contraintes robustes sont violées par les sous-problèmes de séparation (SP1 pour les distances, SP2 pour les poids). Si oui, on ajoute une coupe et on relance la résolution.

Pseudo code

Algorithm 1 Plans Coupants Naïfs

Require: Graphe G , Paramètres K, B, L, W

Ensure: Solution Optimale Robuste $(\hat{x}, \hat{y})$

```

1: Initialiser le Problème Maître ( $MP$ ) avec contraintes statiques et scénarios nominaux
2: while Non Convergé do
3:    $(\hat{x}, \hat{y}, \hat{\eta}) \leftarrow$  Résoudre le  $MP$  à l'optimalité (MIP complet)
4:   Calculer le coût réel maximum via SP1 (Distances) et SP2 (Poids)
5:   if Violation détectée (Coût Réel  $> \hat{\eta}$  OU Poids  $> B$ ) then
6:     Ajouter les contraintes de coupe correspondantes au  $MP$ 
7:   else
8:     Arrêt : La solution courante est robuste optimale
9:     return  $(\hat{x}, \hat{y})$ 
10:  end if
11: end while

```

Analyse

Cette méthode s'est révélée inefficace. Le redémarrage constant du solveur perd l'information de l'arbre de recherche précédent. De plus, elle ne fournit pas de borne inférieure valide ni de solution candidate avant la convergence totale. Elle n'arrive à trouver l'optimum que pour 5 solutions.

Quelques Résultats

Table 3: Résultats Plans Coupants

Instance	Objectif	Gap (%)	Temps (s)
10_ulysses_3.tsp	137.00	0	1.58
10_ulysses_6.tsp	55.12	0	5.10
10_ulysses_9.tsp	33.29	0	1.57
14_burma_3.tsp	93.39	0	6.21
14_burma_6.tsp	∞	-	300.00
14_burma_9.tsp	∞	-	300.00
22_ulysses_3.tsp	358.64	0	149.43
22_ulysses_6.tsp	∞	-	300.00
22_ulysses_9.tsp	∞	-	300.00
26_eil_3.tsp	∞	-	300.00
26_eil_6.tsp	∞	-	300.00
26_eil_9.tsp	∞	-	300.00

2.4.2 Branch-and-Cut (LazyCallback)

L'implémentation via `LazyConstraintCallback` permet d'intégrer la génération de coupes au sein même de l'arbre de recherche de Gurobi. Contrairement à l'approche naïve, le solveur n'est pas redémarré : les coupes de robustesse ne sont ajoutées que lorsque le solveur trouve une solution entière candidate potentielle, agissant comme un filtre de faisabilité dynamique.

Pseudo code

Algorithm 2 Branch-and-Cut (LazyCallback)

Require: Graphe G , Paramètres K, B, L, W

Ensure: Solution Optimale Robuste $(\hat{x}, \hat{y})$

- 1: Initialiser le Modèle (M) avec contraintes statiques uniquement
 - 2: Définir la fonction **Callback** (x^*, y^*, η^*) :
 - 3: Calculer le coût réel via SP1 (Distances) et le poids réel via SP2 (Poids)
 - 4: **if** Violation détectée **then**
 - 5: Ajouter la contrainte de coupe correspondante (Lazy Constraint) au pool global
 - 6: Rejeter la solution candidate
 - 7: **end if**
 - 8: Configurer le solveur pour utiliser **Callback** aux nœuds entiers
 - 9: $(\hat{x}, \hat{y}) \leftarrow$ Lancer l'optimisation (Branch-and-Bound ininterrompu)
 - 10: **return** $(\hat{x}, \hat{y})$
-

Analyse

Bien que supérieure à l'approche naïve (car elle conserve l'arbre de recherche), cette méthode souffre de l'ajout de coupe aux noeuds entier et des difficultés inhérentes du problème statique, ne renvoie ni de bornes inférieures intéressantes ni de solutions heuristiques de bonnes qualités.

Quelques Résultats

Table 4: Résultats Branch and Cut (LazyCallBack)

Instance	Objectif	Gap (%)	Temps (s)
22_ulysses_3.tsp	358.64	0	66.95
22_ulysses_6.tsp	118.10	78	300.23
22_ulysses_9.tsp	64.97	100	300.69
52_berlin_3.tsp	207950.66	95	300.13
52_berlin_6.tsp	∞	-	300.12
52_berlin_9.tsp	∞	-	300.05
318_lin_3.tsp	∞	-	300.08
318_lin_6.tsp	∞	-	300.12
318_lin_9.tsp	∞	-	300.09

2.4.3 Dualisation Compacte

En remplaçant les sous-problèmes de maximisation (le "pire cas") par leurs duals de minimisation directement dans le modèle maître, nous obtenons un Programme Linéaire Mixte (MIP) dont on a tiré notre formulation duale compacte (cf question 4-g)

Analyse

C'est l'approche la plus robuste parmi les méthodes exigées. En fournissant toutes les contraintes simultanément, Gurobi peut appliquer ses propres heuristiques de présolution et ses coupes génériques sur l'ensemble du problème, cela améliore la qualité du gap (qui n'est cependant pas compétitif avec les méthodes combinatoires pour la plupart des instances), la vitesse de résolution et la qualité des solutions renvoyées.

Quelques résultats

2.5 Méthodes Heuristiques : Génération de Colonnes

Face aux limites des modèles compacts, dont la relaxation continue fournit des bornes inférieures de mauvaise qualité, on implémente une heuristique de génération de colonnes basée sur la reformulation de Dantzig-Wolfe (non-compacte). Cette approche vise à obtenir une relaxation plus forte (borne inférieure améliorée) et à casser les symétries inhérentes au modèle initial.

Table 5: Résultats Dualisation Robuste

Instance	Objectif	Gap (%)	Temps (s)
22_ulysses_3.tsp	358.64	0	5.76
22_ulysses_6.tsp	116.53	0	34.82
22_ulysses_9.tsp	64.97	0	6.87
52_berlin_3.tsp	197702.72	67	300.02
52_berlin_6.tsp	92194.67	85	300.06
52_berlin_9.tsp	70531.27	92	300.10
318_lin_3.tsp	30395877.74	99	300.17
318_lin_6.tsp	∞	-	300.00
318_lin_9.tsp	77567343.60	100	300.13

Décomposition de Dantzig-Wolfe

Le problème de partitionnement robuste présente une structure bloc-angulaire naturelle.

- Les contraintes de partitionnement ($\sum_k x_{ik} = 1$) sont des contraintes "liantes" qui couplent les décisions.
- Les contraintes de capacité robuste et de structure interne des clusters sont indépendantes pour chaque sous-ensemble k .

En considérant le modèle robuste dualisé (cf. Section 2), nous pouvons isoler le polytope $\mathcal{P}$ définissant un cluster robuste admissible (un sous-ensemble de sommets respectant la capacité robuste U^2). D'après le théorème de Minkowski-Weyl, tout point de ce polytope peut s'exprimer comme une combinaison convexe de ses points extrêmes (les clusters valides). Comme les K partitions sont identiques (mêmes contraintes, mêmes coûts), nous agrégeons ces sous-problèmes identiques. Le modèle se reformule alors en choisissant K colonnes parmi l'ensemble Ω de toutes les partitions admissibles possibles.

Formulation du Problème Maître Restreint (RMP)

Soit Ω l'ensemble des colonnes (clusters robustes admissibles) générées à une itération donnée. Pour chaque colonne $p \in \Omega$, nous définissons :

- $c_p^{nom} = \sum_{\{i,j\} \in p} l_{ij}$: le coût nominal des arêtes internes au cluster.
- $a_{ip} \in \{0, 1\}$: coefficient valant 1 si le sommet i appartient au cluster p .
- $e_{ij,p} \in \{0, 1\}$: coefficient valant 1 si l'arête $\{i, j\}$ est interne au cluster p .

Les variables globales de robustesse sur les distances (σ^1, μ_{ij}^1) , liées au budget L , restent dans le maître. La variable de décision $\lambda_p \geq 0$ représente la sélection de la colonne p .

$$\begin{aligned}
& \min \sum_{p \in \Omega} c_p^{nom} \lambda_p + L \sigma^1 + \sum_{\{i,j\} \in E} 3 \mu_{ij}^1 \\
& \text{s.c.} \quad \sum_{p \in \Omega} a_{ip} \lambda_p = 1 \quad \forall i \in V \quad (\text{dual } \pi_i) \\
& \quad \sum_{p \in \Omega} \lambda_p = K \quad (\text{dual } \nu) \\
& \quad \sigma^1 + \mu_{ij}^1 - \sum_{p \in \Omega} (\hat{l}_i + \hat{l}_j) e_{ij,p} \lambda_p \geq 0 \quad \forall \{i, j\} \in E \quad (\text{dual } \alpha_{ij}) \\
& \quad \lambda_p \geq 0, \quad \sigma^1 \geq 0, \quad \mu_{ij}^1 \geq 0
\end{aligned}$$

La contrainte liée aux variables duales α_{ij} assure le lien entre les arêtes sélectionnées par les colonnes et le coût robuste global.

2.5.1 Le Sous-Problème (Pricing) et Coût Réduit

Le problème de pricing consiste à trouver une nouvelle colonne p^* (définie par les variables binaires de sommets z_i et d'arêtes y_{ij}) appartenant à l'ensemble des clusters robustes valides, qui minimise le coût réduit $\bar{c}$. En isolant les termes associés à λ_p dans le dual du RMP, l'expression du coût réduit à minimiser est :

$$\bar{c}(z, y) = \underbrace{\sum_{\{i,j\} \in E} l_{ij} y_{ij}}_{\text{Coût nominal}} - \sum_{i \in V} \pi_i z_i - \nu + \sum_{\{i,j\} \in E} \alpha_{ij} (\hat{l}_i + \hat{l}_j) y_{ij}$$

Ce qui se réécrit en regroupant les termes sur les arêtes :

$$\bar{c}(z, y) = \sum_{\{i,j\} \in E} \left(l_{ij} + \alpha_{ij} (\hat{l}_i + \hat{l}_j) \right) y_{ij} - \sum_{i \in V} \pi_i z_i - \nu$$

Contraintes du Pricing : Le cluster doit former une clique locale (les arêtes suivent les sommets) et respecter la capacité robuste locale (U^2) :

- $y_{ij} \geq z_i + z_j - 1$ (Linéarisation)
- $\text{PoidsNominal}(z) + \text{CoûtRobuste}(U^2, z) \leq B$

2.5.2 Algorithme de Résolution et Calcul de Borne

La résolution exacte du pricing est un problème NP-difficile (proche du *Knapsack Problem* quadratique). Pour garantir l'efficacité de la méthode, nous avons implémenté une stratégie de résolution en cascade: dans un premier temps on lance une heuristique gloutonne. Si elle échoue à renvoyer une colonne négative on lance Gurobi avec une time-limit très courte. S'il échoue on utilise le PLNE et Gurobi pour faire une résolution exacte.

Une propriété fondamentale de la décomposition nous permet de calculer une borne inférieure valide (LB_{valid}) à chaque itération ou l'on calcule le cout réduit de plus petite valeur

$$LB_{valid} = Z_{RMP} + K \times \min(0, \bar{c}_{min})$$

Algorithm 3 Génération de Colonnes avec Pricing Hybride

Require: Graphe G , Paramètres K, B, L, W **Ensure:** Borne Inférieure (LB) et Solution Entière (UB)

```
1: Initialiser RMP avec une population de colonnes (Heuristique Géométrique)
2:  $LB_{best} \leftarrow -\infty$ 
3: while Temps < TimeLimit do
4:   Résoudre la relaxation linéaire du RMP  $\rightarrow$  Duaux  $\pi, \nu, \alpha$ 
5:    $Z_{RMP} \leftarrow$  Objectif du RMP
6:   ColonneTrouvee  $\leftarrow$  Faux
7:   // Étape 1 : Pricing Heuristique (Rapide)
8:    $(Col, \bar{c}) \leftarrow$  HeuristiqueGlouton + RechercheLocale( $\pi, \nu, \alpha$ )
9:   if  $\bar{c} < -\epsilon$  then
10:    Ajouter Col au RMP, ColonneTrouvee  $\leftarrow$  Vrai
11:   end if
12:   if Non ColonneTrouvee then
13:    // Étape 2 : Pricing MIP Restreint (Moyen)
14:     $(Col, \bar{c}) \leftarrow$  SolveMIP( $\pi, \nu, \alpha$ ) avec TimeLimit = 2s
15:    if  $\bar{c} < -\epsilon$  then
16:      Ajouter Col au RMP, ColonneTrouvee  $\leftarrow$  Vrai
17:    end if
18:   end if
19:   if Non ColonneTrouvee then
20:    // Étape 3 : Pricing Exact (Lent - Preuve d'optimalité)
21:     $(Col, \bar{c}) \leftarrow$  SolveMIP( $\pi, \nu, \alpha$ ) sans limite
22:    if  $\bar{c} < -\epsilon$  then
23:      Ajouter Col au RMP
24:      // Mise à jour de la Borne Inférieure
25:       $LB_{curr} = Z_{RMP} + K \times \bar{c}$ 
26:       $LB_{best} = \max(LB_{best}, LB_{curr})$ 
27:    else
28:      Break (Optimalité de la relaxation atteinte)
29:    end if
30:   end if
31: end while
32: // Phase Finale : obtention d'une solution entière
33: Résoudre le RMP avec  $\lambda_p \in \{0, 1\}$  sur les colonnes générées
34: return  $LB_{best}$ , Solution Entière
```

2.6 Stratégie d'Initialisation et Heuristique de Recherche Locale

Pour amorcer efficacement le problème maître (RMP) de la génération de colonnes avec des partitions pertinentes, une stratégie heuristique en deux phases a été mis en place exploitant la géométrie du graphe.

Phase 1 : Initialisation Géométrique (K-Means)

Puisque les coûts l_{ij} correspondent à des distances euclidiennes, une partition minimisant la somme des distances intra-classes est proche d'un clustering K-mean (qui minimise la norme au carré au lieu de la norme). On initialise la solution en appliquant l'algorithme des K -moyennes sur les coordonnées des sommets. Cette approche fournit quasi-instantanément une partition géométriquement cohérente, bien que potentiellement infaisable vis-à-vis des contraintes de capacité robustes B .

Phase 2 : Réparation Relaxée par Score Composite

Pour recouvrer l'admissibilité tout en optimisant le coût, on utilise une recherche locale basée sur une relaxation dynamique. On définit un score composite pénalisant la violation des capacités robustes :

$$\text{CoûtRobuste} + \lambda \times \sum_{k=1}^K \max(0, \text{PoidsRobuste} - B)$$

L'algorithme procède par descente locale en évaluant deux types de voisinages

- **Shift** : Déplacement d'un sommet u d'une partition V_i vers une partition V_j .
- **Swap** : Échange de deux sommets $u \in V_i$ et $v \in V_j$.

Le paramètre de pénalité λ est ajusté dynamiquement : il est augmenté lorsque la solution est infaisable (pour forcer le respect de B) et diminué lorsque la solution est admissible (pour privilégier la minimisation du coût U^1)

Utilisation en tant que solveur heuristique. Au-delà de l'initialisation de la génération de colonnes, cette méthode peut aussi être utilisée comme un solveur heuristique indépendants. Comparée aux bornes inférieures calculées dans les sections précédentes, cette méthode sera analysée dans la section suivante.

2.7 Analyse

La méthode de génération de colonnes surpasse les approches compactes sur les instances difficiles.

- La relaxation continue du RMP fournit une borne inférieure (LB_{colgen}) plus forte que celle du modèle compact.
- L'heuristique de pricing permet de générer de nombreuses colonnes pertinentes très rapidement.

Malheureusement la fermeture de l'instance nécessite la résolution de plusieurs instances du problème de pricing à l'exactitude ce qui est très coûteux (voir impossible pour les plus grosses instances). La limite de temps de 300s sur la méthode ne lui permet pas de converger et on voit une net baisse de la qualité de la borne inférieure au fur et à mesure que les instances grossissent. Ils restent donc plus pertinents pour la plupart des instances de comparer la valeur de l'objectif obtenue avec la borne robuste obtenue par la relaxation par pseudo-clique. Néanmoins, sous réserve de pouvoir fermer l'instance, les bornes renvoyées par génération de colonnes sont plus intéressantes que les bornes combinatoires établies avant

Quelques Résultats

Table 6: Résultats Génération Colonne

Instance	Objectif	Gap (%)	Temps (s)
22_ulysses_3.tsp	363.35	3	14.37
22_ulysses_6.tsp	117.60	5	2.93
22_ulysses_9.tsp	64.97	1	2.78
52_berlin_3.tsp	188965.50	19	300.33
52_berlin_6.tsp	75620.02	26	300.31
52_berlin_9.tsp	58699.84	47	301.48
318_lin_3.tsp	20285837.86	771	439.57
318_lin_6.tsp	6752187.47	∞	361.25
318_lin_9.tsp	3739114.69	∞	338.09

2.7.1 Méthode Relaxed And Repair

Comme mentionné on peut utiliser la méthode heuristique de peuplement de colonnes et comparer la solution obtenue à une des bornes inférieures combinatoires pour obtenir une méthode qui est :

- Plus rapide sur la plupart des instances
- Avec garantie de performance

C'est cette méthode qui obtient les meilleurs résultats, cependant elle a eu un budget de temps plus élevé.

Table 7: Performance Heuristique Robuste avec Borne Pseudo-Clique (Sélection)

Instance	Objectif	Gap (%)	Temps Total (s)
22_ulysses_3.tsp	358.64	22	2.50
22_ulysses_6.tsp	156.98	50	11.39
22_ulysses_9.tsp	64.97	48	8.51
52_berlin_3.tsp	171519.27	18	42.73
52_berlin_6.tsp	71874.47	30	129.20
52_berlin_9.tsp	53672.45	43	264.26
318_lin_3.tsp	20857632.01	26	436.05
318_lin_6.tsp	6982427.66	27	470.82
318_lin_9.tsp	3440147.26	21	533.16

Quelques Résultats

2.8 Comparaison de toutes les méthodes

3 Conclusion

Ce projet a permis de tirer différents enseignements quant à la manière de traiter ce problème de partitionnement de graphe.

- Premièrement, les formulations compactes atteignent rapidement un plafond de performance. Si la **dualisation** du problème robuste s'est avérée supérieure aux méthodes itératives de plans coupants (naïfs ou via *LazyCallback*), elle reste limitée par la faiblesse de la relaxation continue et les symétries du modèle, ne permettant pas de résoudre efficacement des instances au-delà de 50 sommets.
- Deuxièmement, une heuristique non compacte comme la génération de colonnes permet de résoudre les problèmes des méthodes compactes mais échoue à converger sur les plus grosses instances due à la difficulté du problème de Pricing
- Finalement, la méthode heuristique Relaxed and Repair avec Gap Combinatoire s'impose comme la stratégie la plus performante. Elle trouve les meilleures solutions, le meilleur gap en le moins de temps.

Néanmoins plusieurs pistes d'amélioration de l'algorithme de génération de colonnes mériteraient d'être explorées pour le rendre compétitif :

- **Amélioration du Pricing** : Le sous-problème étant lui-même NP-difficile, l'implémentation d'heuristiques plus sophistiquées (métaheuristiques, programmation dynamique, relaxation) pour la résolution du *pricing* permettrait d'accélérer la convergence en générant des colonnes de meilleure qualité plus rapidement.
- **Inégalités Valides** : Pour combler le gap d'intégralité restant sur les instances les plus difficiles (K grands et non fermeture de la génération de colonnes), l'ajout d'inégalités valides spécifiques au problème maître serait nécessaire pour renforcer la formulation.
- **Utilisation dans un Brach and Price** pour chercher des solutions exactes

Table 8: Comparaison Globale des Méthodes (Complet)

Instance	PoR (%)	Plans Coup.		Branch & Cut		Dualisation		Gén. Col.		Heuristique	
		Gap	T(s)	Gap	T(s)	Gap	T(s)	Gap	T(s)	Gap	T(s)
10_ulysses_3.tsp	60	0	1.6	0	0.4	0	0.1	0	0.9	40	0.5
10_ulysses_6.tsp	71	0	5.1	0	1.6	0	0.2	24	0.3	11	3.3
10_ulysses_9.tsp	96	0	1.6	0	0.7	0	0.1	77	0.3	14	3.6
14_burma_3.tsp	28	0	6.2	0	1.8	0	0.4	1	1.1	34	0.8
14_burma_6.tsp	55	-	300.0	0	39.9	0	0.5	12	0.5	28	4.9
14_burma_9.tsp	70	-	300.0	100	300.2	0	0.4	37	0.2	1	4.5
22_ulysses_3.tsp	19	0	149.4	0	67.0	0	5.8	3	14.4	22	2.5
22_ulysses_6.tsp	28	-	300.0	78	300.2	0	34.8	5	2.9	50	11.4
22_ulysses_9.tsp	54	-	300.0	100	300.7	0	6.9	1	2.8	48	8.5
26_eil_3.tsp	19	-	300.0	52	300.3	0	184.5	3	44.9	19	32.9
26_eil_6.tsp	38	-	300.0	88	300.3	41	300.3	18	4.6	23	18.9
26_eil_9.tsp	57	-	300.0	-	300.5	35	300.1	31	7.4	24	14.5
30_eil_3.tsp	11	-	-	52	300.2	20	300.8	1	166.2	21	5.6
30_eil_6.tsp	30	-	-	90	300.0	47	300.1	16	28.3	27	26.7
30_eil_9.tsp	29	-	-	100	300.2	32	300.1	8	8.0	30	17.5
34_pr_3.tsp	28	-	-	67	300.0	37	300.1	14	300.1	24	11.4
34_pr_6.tsp	40	-	-	92	300.1	54	300.1	20	80.7	30	54.1
34_pr_9.tsp	52	-	-	-	308.5	68	300.2	34	19.8	26	36.1
38_rat_3.tsp	16	-	-	72	300.2	43	300.2	18	300.2	20	96.8
38_rat_6.tsp	47	-	-	95	300.2	71	300.1	27	300.7	27	115.7
38_rat_9.tsp	57	-	-	-	304.3	84	300.0	35	67.8	24	272.3
40_eil_3.tsp	13	-	-	79	300.2	59	300.2	17	300.2	18	109.0
40_eil_6.tsp	35	-	-	-	300.1	77	300.1	21	300.1	28	16.5
40_eil_9.tsp	47	-	-	-	301.5	80	300.0	26	51.5	31	45.9
44_lin_3.tsp	24	-	-	89	300.2	62	300.0	30	300.1	27	149.1
44_lin_6.tsp	47	-	-	-	300.1	75	300.0	36	300.3	27	200.6
44_lin_9.tsp	50	-	-	-	301.3	86	300.0	32	123.3	22	300.7
48_att_3.tsp	8	-	-	86	300.3	48	300.2	56	300.1	15	154.0
48_att_6.tsp	30	-	-	-	300.1	79	300.0	76	300.3	43	97.7
48_att_9.tsp	37	-	-	-	303.1	89	300.1	29	108.7	37	55.8
52_berlin_3.tsp	8	-	-	95	300.1	67	300.0	19	300.3	18	42.7
52_berlin_6.tsp	22	-	-	-	300.1	85	300.1	26	300.3	30	129.2
52_berlin_9.tsp	53	-	-	-	300.0	92	300.1	47	301.5	43	264.3
70_st_3.tsp	8	-	-	-	300.0	83	300.0	-	300.6	15	363.7
70_st_6.tsp	26	-	-	-	300.7	88	300.4	101	300.3	27	68.1
70_st_9.tsp	32	-	-	-	304.7	95	300.0	47	300.3	28	156.9
80_gr_3.tsp	9	-	-	-	300.0	78	300.0	-	301.6	20	468.0
80_gr_6.tsp	21	-	-	-	302.3	90	300.0	-	300.3	25	141.4
80_gr_9.tsp	41	-	-	-	316.2	93	300.0	55	300.3	34	239.1
100_kroA_3.tsp	8	-	-	-	300.0	89	300.0	-	307.0	17	190.0
100_kroA_6.tsp	11	-	-	-	300.7	95	300.0	-	300.8	23	571.0
100_kroA_9.tsp	24	-	-	-	300.1	99	300.1	120	300.4	27	422.7
202_gr_3.tsp	7	-	-	100	300.1	97	300.2	336	334.0	33	400.5
202_gr_6.tsp	11	-	-	-	300.1	100	300.2	-	320.2	30	408.6
202_gr_9.tsp	18	-	-	-	305.3	100	300.2	-	304.0	35	448.5
318_lin_3.tsp	9	-	-	-	300.1	99	300.2	771	439.6	26	436.0
318_lin_6.tsp	14	-	-	-	300.1	-	300.0	-	361.2	27	470.8
318_lin_9.tsp	15	-	-	-	300.1	100	300.1	-	338.1	21	533.2
400_rd_3.tsp	7	-	-	-	300.1	100	300.1	954	373.6	24	442.9
400_rd_6.tsp	10	-	-	-	300.1	100	300.1	1495	432.5	32	445.3
400_rd_9.tsp	17	-	-	-	300.3	-	300.0	-	367.8	39	457.7
532_att_3.tsp	28	-	-	-	346.9	100	302.6	-	-	42	458.7
532_att_6.tsp	26	-	-	-	357.8	100	300.2	-	-	100	480.6
532_att_9.tsp	-	-	-	-	35.5	-	300.0	-	-	-	-